Day 1: How big is the QR, and how was it written?

Simple notes for today's work: checking the size of the QR code, and reading the small "settings" label hidden inside it. They also explain what each piece of your Python code does.

1 size → version | **2 read the settings** | 3 read data bits | 4 sort into blocks | 5 fix errors | 6 turn bits into text | 7 send bytes out

Today covers the first two steps. Nothing else in the decoder can work until these two are right.

---

1. The big picture

A QR code is a square grid of small black and white squares. Each square is called a **module**. Black = 1, white = 0.

Before the decoder can read the message, it needs to know three things:

- **The version (1 to 40):** how big the grid is.
- **The error-correction level (L, M, Q or H):** how much of the code is spare "backup" data for fixing damage.
- **The mask (0 to 7):** which pattern was used to scramble the data (explained in section 4).

The **version** comes from the size of the grid. The **level and mask** come from a small label of 15 squares inside the code, called the **format information**.

---

2. Step 1: How big is the QR code? (the version)

Every version is 4 squares wider than the one before. Version 1 is 21 × 21, version 2 is 25 × 25, and so on up to version 40, which is 177 × 177. As a formula: `side = 17 + 4 × version`.

The test bench only tells us the side length. So we work backwards: `version = (side − 17) ÷ 4`. If that doesn't give a whole number from 1 to 40, the size is impossible and the decoder must **fail**.

Side given	Working	Answer
21	`(21 − 17) ÷ 4 = 1`	version 1
177	`(177 − 17) ÷ 4 = 40`	version 40
22	`(22 − 17) ÷ 4 = 1.25`	**fail (not a whole number)**
17	`(17 − 17) ÷ 4 = 0`	**fail (there is no version 0)**
181	`(181 − 17) ÷ 4 = 41`	**fail (there is no version 41)**

Why we check "is it between 21 and 177?" first

In the hardware the size is stored in 8 bits, so it can never be negative. If the size were 5, then 5 − 17 would not give −12. It would wrap around to a big number like 244. Checking the range first avoids that trap.

---

3. Step 2: The settings label (format information)

The settings are only **5 bits**: 2 bits for the level and 3 bits for the mask.

d4	d3	d2	d1	d0
level	level	mask	mask	mask

Level bits	Level	How much backup data
`01`	L	least (about 7%)
`00`	M	about 15%
`11`	Q	about 25%
`10`	H	most (about 30%)

Watch out

The order is **not** alphabetical. M is `00` and L is `01`. Our code always keeps the raw 2 bits (called `level_field`) so they can't get mixed up.

The sample QR code uses level M (`00`) and mask 2 (`010`), so its 5 bits are `00010`.

These 5 bits are very important. If they are read wrongly, the whole message is read wrongly. So the QR code protects them in two ways:

1. It adds **10 extra check bits**, making 15 bits in total. These let us fix up to 3 wrong bits.
2. It stores the 15 bits **twice**, in two different places.

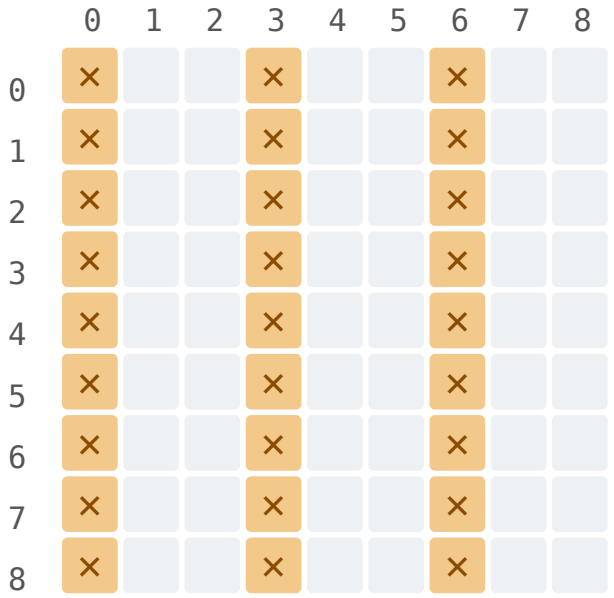
---

4. What is a mask?

Sometimes the message bits come out in an ugly shape, such as a big white patch or a big black blob. Phone cameras find those hard to read. So the encoder **flips** some of the squares using a simple fixed pattern. That pattern is the **mask**.

Example: mask 2 means "flip every square whose column number divides by 3": columns 0, 3, 6, 9, and so on. (× = flipped.)

- **When making the QR:** black becomes white and white becomes black in those columns.
- **When reading the QR:** we flip the same squares back. Flipping twice gives the original.
- There are 8 masks (0 to 7), each a different simple rule (spec Table 2). The 3 mask bits tell us which one was used.
- Only **data** squares are flipped. The fixed patterns (corner squares, timing lines, settings label) are never flipped.



Example from the sample: square (row 20, column 18) is stored as black (1). Column 18 divides by 3, so it was flipped, and the real bit is 0. Square (20, 20) is stored as white (0). Column 20 does not divide by 3, so it stays 0.

---

5. Bits and XOR: the only maths you need today

XOR compares two bits: the answer is 1 if they are **different** and 0 if they are the **same**.

```
0 XOR 0 = 0     0 XOR 1 = 1     1 XOR 0 = 1     1 XOR 1 = 0
```

With these special codes we treat XOR as both "add" and "subtract", and nothing is ever carried. That makes the division in the next section simple: each step is just one XOR.

XOR is also the "flip" from the mask section: XOR with 1 flips a bit, and XOR with 0 leaves it alone.

---

6. How 5 bits become 15 bits

The spec gives a fixed recipe. It uses a fixed 11-bit number called the **generator**: `10100110111`.

1. Make room
Add 10 zeros after the 5 bits.

2. Divide
Divide by the generator using XOR, and keep the 10-bit remainder.

3. Join
Put the 5 bits in front of the 10-bit remainder: 15 bits.

4. Disguise
XOR with `101010000010010` so the label is never all white.

Worked example: the sample (5 bits = 00010)

```
step 1  add 10 zeros          000100000000000
step 2  a 1 is at position 11, so XOR the generator lined up under it:
                              000101001101110
        result                000001001101110   ← first 5 positions are now 0, stop
        remainder (10 bits)       1001101110
step 3  join                  000101001101110
step 4  XOR disguise          101010000010010
        stored in the QR      101111001111100   ← this is exactly what the sample holds
```

There are only 32 possible 5-bit settings, so there are only **32 valid 15-bit labels**. The course file `QRTables.bsv` lists the same 32, and our test checks that we match every one.

---

7. Reading the label when some squares are damaged

A dirty or scratched QR may have some squares wrong. So when we read the 15 bits, we ask: **"Which of the 32 valid labels is closest to what we read?"**

Distance = how many bits are different

```
what we read   101111001111011
valid label    101111001111100
different?     000000000000111   → 3 bits differ → distance = 3
```

In code: XOR the two numbers, then count the 1s.

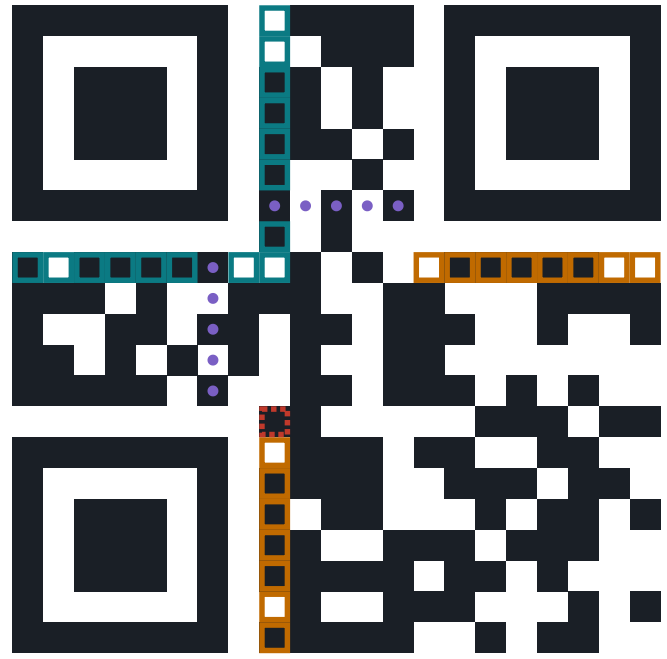
Why up to 3 wrong bits can be fixed

The 32 valid labels were chosen so that **any two of them differ in at least 7 bits**. Think of each valid label as a house, with a "safe zone" around it of everything within 3 bits:

- Two houses are at least 7 steps apart. Each safe zone reaches only 3 steps, and 3 + 3 = 6, which is less than 7. So **the safe zones never touch**.
- If at most 3 bits were damaged, what we read is still inside the right house's safe zone, so the closest label is the correct one.
- If 4 or more bits were damaged, what we read might be closer to the **wrong** house. If the closest label is more than 3 away, we can't trust it, and the decoder must **fail**.

> **A number we can test**
>
> One safe zone holds 1 (no errors) + 15 (1 error) + 105 (2 errors) + 455 (3 errors) = 576 patterns. 32 labels × 576 = **18,432**. So out of all 32,768 possible 15-bit readings, a correct decoder accepts exactly 18,432. Our test checks this number.

8. The two copies, and how we pick one



copy 1 copy 2 timing line (row/col 6)
always-black square (not part of the label)

Copy 1 wraps around the top-left corner square. It goes down column 8, then along row 8.

Copy 2 is split in two: 8 bits along row 8 near the top-right, and 7 bits down column 8 near the bottom-left.

Copy 1 **jumps over** row 6 and column 6. Those hold the timing line, a fixed black-white-black pattern.

Keeping two copies far apart means one smudge can't destroy both.

The rule for choosing (spec G2)

1. Find how far each copy is from its closest valid label: d1 and d2.
2. If copy 2 is **strictly** closer (d2 < d1), use copy 2.
3. Otherwise (copy 1 closer, or **a tie**), use copy 1.
4. If the chosen distance is more than 3, **fail**.

Five examples on the sample (worked out by the code)

What we damaged	Copy 1 distance	Copy 2 distance	Which copy is used	Result
nothing	0	0	copy 1 (tie)	**M, mask 2 ✓**
3 bits in copy 1, 4 in copy 2	3	4	copy 1 (closer)	**M, mask 2 ✓**
2 bits in each copy	2	2	copy 1 (tie)	**M, mask 2 ✓**
4 bits in copy 1 only	3, but it points to the **wrong** mask (3)	0	copy 2 (closer)	**M, mask 2 ✓**
4 bits in each copy	4	4	copy 1 (tie)	**fail (more than 3)**

The fourth row is the interesting one. With 4 errors, copy 1 alone gives the **wrong answer** (mask 3), but it looks only 3 away. Because copy 2 is clean and closer, the decoder still gets the right answer. This is why we compare both copies and don't just take copy 1 whenever it looks fixable.

9. The code, explained simply

File: `verification/model/front_ref.py`. This Python is our **reference**: the "answer key" that Rishi's hardware will be checked against. It is written to be easy to read and obviously right, not fast.

version_from_size(n): is the size valid, and which version?

```
if n < 21 or n > 177:   return None     # too small or too big → invalid
if (n - 17) % 4 != 0:    return None     # not on the 17 + 4v ladder → invalid
return (n - 17) // 4                     # the version number
```

`None` means "invalid size, fail". `%` gives the remainder of a division and `//` divides without decimals.

format_positions(n): where are the 15 label squares?

This gives two lists of 15 (row, column) places, one for each copy. Place number `i` holds bit `f_i`. Each line of code copies one row of spec Table 1, so you can check them side by side.

bch_encode(data5): turn 5 bits into the 15-bit label

```
rem = data5 << 10                           # step 1: add 10 zeros
for bit in range(14, 9, -1):                # look at positions 14, 13, 12, 11, 10
    if (rem >> bit) & 1:                    # is there a 1 here?
        rem ^= FORMAT_GENERATOR << (bit - 10)   # step 2: XOR the generator under it
codeword = (data5 << 10) | rem              # step 3: join
return codeword ^ FORMAT_XOR_MASK           # step 4: disguise
```

`<< 10` shifts bits left by 10 places (adds 10 zeros). `>> bit` then `& 1` reads one bit. `^` is XOR and `|` joins bits together. It is exactly the worked example in section 6.

valid_format_words(): list all 32 valid labels

Runs `bch_encode` for every setting from 0 to 31. It stores each label with its meaning: the level (top 2 bits, `data5 >> 3`) and the mask (bottom 3 bits, `data5 & 0b111`). This is done once, when the file loads.

read_format(matrix, n): read both copies from the QR

For each of the 15 places, it looks up the square (1 or 0) and puts it at bit position `i` of a number. The result is two 15-bit numbers: copy 1 and copy 2.

hamming(a, b) and nearest_format(word): find the closest valid label

```
hamming(a, b) = count of 1s in (a XOR b)          # how many bits differ
nearest_format: try all 32 valid labels, keep the one with the smallest distance
```

Trying all 32 every time is slow but very safe, which is what we want in an answer key. It returns three things: the distance, the level and the mask.

decode_format(copy1, copy2): the final decision (rule G2)

```
if d2 < d1:  use copy 2       # copy 2 must be STRICTLY closer
else:        use copy 1       # copy 1 closer, or a tie
if distance > 3: return None  # can't be trusted → fail
```

The single `<` sign is the tie rule. Writing `<=` would wrongly give ties to copy 2.

load_tb_symbol and the demo at the bottom

These read the sample QR straight out of `TbSample.bsv` and print the result:

```
size N = 21  ->  version 1
copy 1: f14..f0 = 101111001111100  nearest distance 0  -> level M (00), mask 2
copy 2: f14..f0 = 101111001111100  nearest distance 0  -> level M (00), mask 2
decision: level M, mask 2
```

---

10. The tests: how we know the code is right

Important rule: **a test must not take its correct answer from the code it is testing.** Otherwise a mistake would just agree with itself. Each test gets its answer from somewhere else:

Test	What it checks	Where the right answer comes from
version_all_sizes	every size from 0 to 255	the formula 17 + 4v
format_positions_shape	15 different places per copy, no overlap, never the timing line	the spec's drawing rules
bch_matches_supplied_table	all 32 labels	the course's own QRTables.bsv file
bch_min_distance_is_7	labels differ in at least 7 bits	the spec
nearest_recovers_up_to_3	every label with 0 to 3 damaged bits is fixed	the safe-zone idea (section 7)
exhaustive_single_copy_count	exactly 18,432 of 32,768 readings accepted	counting (section 7)
two_copy_rules	closer copy wins, tie → copy 1, both bad → fail	the spec's rule G2
sample_format_is_M_mask2	the real sample gives M, mask 2	the handout README

Today's result (run in my workspace)

8 of 14 checks pass. The other 6 are marked TODO because they test tomorrow's work. Run it yourself on the server with `python3 verification/tests/test_front_ref.py`, and save your own output as your evidence.

---

11. Quick questions (cover the answer, try first)

1. Why fail when the closest label is 4 bits away, instead of just using it?

Because 4 bits from the right label can be only 3 bits from a wrong one. We could pick the wrong mask and read the whole message wrongly. Failing is safer, and the spec requires it.

---

2. Both copies are 2 bits away but give different settings. Which wins?

Copy 1 (tie rule). This can only happen if the QR is badly damaged: at least one copy has 5 or more wrong bits.

---

3. Why does copy 1 skip row 6 and column 6?

Those squares hold the timing line, a fixed pattern. They can't also hold label bits.

---

4. What is a mask, in one sentence?

A fixed flip pattern laid over the data squares to avoid ugly shapes. The reader flips the same squares back to recover the real bits.

---

5. The hardware can read one square per clock tick, and each answer arrives one tick later. How many ticks to read both copies?

30 squares in 30 ticks, plus 1 tick for the last answer: at least 31 ticks.
